PYTHON PARA VIDEOJUEGOS CON PYGAME · LECCIÓN 5

Choques, puntos y vidas

Objetivo: Al terminar puedes detectar choques entre Rects, sumar puntos, mostrar texto en pantalla y quitar vidas.
Antes de empezar:

- Lección 4 - los enemigos que caen.

Para copiar el código: al copiar desde un PDF, muchos visores pierden los espacios del principio de cada línea, y en Python esos espacios importan. En la misma carpeta de esta guía está cada paso como archivo `.py`: ábrelo y copia de ahí.

Paso a paso

Choques, puntos y vidas

Hasta ahora las balas atraviesan a los enemigos; hoy eso se acaba. Vas a detectar cuándo una bala toca a un enemigo. Vas a sumar puntos, y mostrarlos en la pantalla. Y vas a perder vidas cuando un enemigo se escapa o te choca.

Collidelist dice si un Rect toca a alguno de una lista

Un choque es solo esto: dos rectángulos que se tocan. Primero, debajo del sonido del disparo, carga hit_sound, el sonido que va a sonar cuando una bala acierta. Después de crear el grupo de enemigos, agrega la variable score, en cero. Ahí se van a ir sumando los puntos de toda la partida. Y después de mover a los enemigos, recorre el grupo con un for, y pregúntale a cada uno si su Rect toca alguna bala. collidelist responde con la posición de esa bala en la lista, o con menos uno si no toca ninguna; por eso el if compara con menos uno. Si acierta, bullets.pop(hit) quita esa bala, para que no cuente dos veces. Reset manda al enemigo arriba, sumas diez puntos, y suena el golpe.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-1.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
```

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
```

```
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Los aciertos hacen desaparecer al enemigo

Ejecuta el juego. Ahora cada bala que acierta hace desaparecer al enemigo, y suena el golpe. Los puntos ya se suman, en la variable score, pero todavía no se ven.

Pruébalo:

```
python game.py
```

El texto se convierte en imagen con font.render

Para escribir en la pantalla, primero necesitas una fuente. pygame.font.Font(None, 36) la crea una sola vez, al principio: None significa la fuente que trae pygame, y treinta y seis es el tamaño. Al final del dibujo, antes de display.flip, font.render convierte el texto en una imagen, porque pygame no dibuja texto directamente. La f delante de las comillas y las llaves alrededor de score meten su valor dentro del texto, True suaviza los bordes, y el último valor es el color blanco. Con blit pegas esa imagen arriba a la izquierda, en cada cuadro.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-2.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
font = pygame.font.Font(None, 36)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)
```

```
    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Colliderect compara con un solo Rect; chocar o dejar pasar cuesta una vida

Ahora, el peligro. Debajo del sonido del golpe, carga lose_sound, el sonido de perder una vida. Va a sonar cada vez que un enemigo te gane. En update, borra las dos líneas que mandaban al enemigo arriba cuando salía por abajo. Si update lo reiniciara, el bucle nunca lo vería escaparse, y escaparse ahora cuesta una vida. Debajo de score, agrega lives, en tres. Son las vidas con las que empieza cada partida. Y en los choques, agrega un elif: si el enemigo no recibió una bala, pero pasó por abajo, o su Rect toca al ovni, vuelve arriba, pierdes una vida, y suena el sonido nuevo. Para comparar con un solo Rect, como el del ovni, usas colliderect, que responde verdadero o falso.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-3.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
```

```
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()
        elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
            enemy.reset()
            lives -= 1
            lose_sound.play()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Sin vidas, el juego termina

Falta el final. Después de los choques, fuera del for, pregunta si las vidas llegaron a cero, o a menos de cero, que puede pasar si dos enemigos te quitan vidas en el mismo cuadro. Si es así, running pasa a False, el bucle termina, y la ventana se cierra. Y el marcador ahora muestra también las vidas. En el mismo texto con f, después del puntaje, agregas la palabra Lives y el valor de la variable lives, así ves cuántas te quedan.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-4.py`):

```
import random

import pygame
```

```
pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
```

```
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()
        elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
            enemy.reset()
            lives -= 1
            lose_sound.play()
    if lives <= 0:
        running = False

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

El juego ya tiene reglas

Ejecuta el juego. Mira el marcador: los puntos suben con cada acierto. Si un enemigo se escapa por abajo, suena otro sonido, y baja una vida. Ya es un juego de verdad: tiene reglas, puntos, y una forma de perder.

Pruébalo:

```
python game.py
```

Lo que aprendiste

Repasemos. Un choque es dos Rects que se tocan. collidelist busca en una lista, y colliderect compara con un solo Rect. font.render convierte el texto en una imagen. Y las vidas le dan al juego un final. Tu ejercicio: cuando llegues a cien puntos, haz que los enemigos caigan más rápido. En la próxima lección, el juego tiene menú, pausa, y pantalla de fin.

- Un choque es dos Rects que se tocan
- collidelist y colliderect
- font.render convierte texto en imagen
- Las vidas llevan al final del juego

Resumen

- `enemy.rect.collidelist(bullets)` devuelve el índice de la primera bala que toca, o `-1` si no toca ninguna.
- `enemy.rect.colliderect(player)` es `True` si los dos `Rect` se superponen.
- `bullets.pop(hit)` quita de la lista la bala que acertó.
- `font = pygame.font.Font(None, 36)` crea una fuente; `font.render(texto, True, color)` devuelve una imagen con el texto.
- `f"Score: {score}"` es un f-string: mete el valor de la variable dentro del texto.
- Cuando `lives` llega a 0, `running = False` termina el bucle.

Ejercicio

Parte del `game.py` de esta lección.

- Cuando `score` llegue a 100 o más, haz que los enemigos nuevos caigan con velocidad 2 a 4 en vez de 1 a 2. Pista: `reset` puede leer la variable global `score`. Ojo: `score` tiene que existir antes de crear los enemigos.
- Muestra las vidas con corazones: dibuja `assets/Life.png` una vez por cada vida, en vez del texto `Lives`.
- Bonus: cada 5 aciertos seguidos sin perder una vida, suma 50 puntos extra.

Solución

```
life_image = pygame.image.load("assets/Life.png").convert_alpha()

score = 0   # muévelo ANTES de esta línea:
enemies = pygame.sprite.Group(Enemy() for _ in range(8))

# en Enemy.reset:
self.speed = random.randint(2, 4) if score >= 100 else random.randint(1, 2)

# al dibujar, en lugar de las vidas en texto:
hud = font.render(f"Score: {score}", True, (255, 255, 255))
screen.blit(hud, (10, 10))
for i in range(lives):
    screen.blit(life_image, (10 + i * 26, 44))
```

En el `game.py` de esta lección, `score = 0` está después del `Group`. Muévelo antes: al crear cada `Enemy` se llama a `reset`, que ya lee `score`.

Bonus:

```
streak = 0                  # junto a score y lives
# al acertar:
streak += 1
if streak % 5 == 0:
    score += 50
# al perder una vida:
streak = 0
```

Gráficos y sonidos: Most Wanted UFO Classic, de Jose Daniel Soto, licencia CC BY 4.0.